Programmation système et scripts shell

Informatique — Licence 2 (L2)

Cours INF-L2-15

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Passer des commandes isolées aux programmes shell : variables, conditions et boucles en bash, scripts d'administration sur les fichiers et les comptes, pilotage des processus, expressions régulières et traitement de texte avec grep, sed et awk, sauvegardes automatisées par cron, puis sécurisation et journalisation des scripts, jusqu'à une boîte à outils d'administrateur complète.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Le shell : variables, conditions et boucles
• Leçon 2 — Scripts d'administration : fichiers et utilisateurs
• Leçon 3 — Gestion des processus en ligne de commande
• Leçon 4 — Expressions régulières et traitement de texte avec grep, sed et awk
• Leçon 5 — Planification et automatisation des sauvegardes
• Leçon 6 — Sécuriser ses scripts et journaliser les actions
• Leçon 7 — Mini-projet : boîte à outils d'administrateur système
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Dans le cours d'administration Linux, tu as appris à taper les bonnes commandes au bon moment. Mais un administrateur ne peut pas rester assis devant son serveur à 2 heures du matin pour lancer la sauvegarde, ni recopier cinquante fois la même suite de commandes pour créer les comptes d'une nouvelle promotion. Dès qu'une tâche se répète, elle mérite d'être écrite une fois pour toutes dans un script shell : un fichier texte contenant des commandes, des variables, des décisions et des boucles, que la machine exécutera sans fatigue et sans faute de frappe. Ce cours t'apprend à programmer le shell bash comme un vrai langage, pas seulement à aligner des commandes. Le fil rouge du cours est toujours notre serveur d'entraînement srv-iam, sur lequel nous écrirons des scripts de plus en plus complets, jusqu'à la boîte à outils finale.

Programmer en shell a ses pièges propres, et ce cours les affronte dès le début : un espace de trop autour du signe = et l'affectation échoue ; une variable non entourée de guillemets et un nom de fichier contenant un espace casse le script ; un code de retour non testé et une erreur passe inaperçue pendant des semaines. Chaque script présenté dans les leçons a donc été réellement exécuté sous bash avant d'être écrit ici, et les sorties affichées sont celles obtenues pendant ces tests — quand un résultat dépend de la machine ou de l'instant (un numéro de processus, une date), le texte le signale. Reproduis chaque exemple sur ta machine virtuelle : en programmation système, on ne croit que ce que le terminal affiche.

La démarche du cours suit la progression d'un administrateur qui automatise son quotidien. D'abord le langage lui-même : variables, conditions, boucles. Puis les deux grands terrains d'application : les fichiers et les comptes d'une part, les processus d'autre part. Vient ensuite l'arme favorite de l'administrateur, le traitement de texte — expressions régulières, grep, sed et awk — car presque tout, sous Linux, est texte : journaux, configurations, listes d'utilisateurs. Nous planifierons alors des sauvegardes automatiques avec cron, nous apprendrons à écrire des scripts sûrs qui s'arrêtent proprement à la première erreur et qui journalisent leurs actions, et nous assemblerons enfin tous ces morceaux dans le mini-projet : une boîte à outils d'administration que tu pourras continuer à enrichir pendant ton stage.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Écrire un script bash structuré avec variables, paramètres, conditions (if, case) et boucles (for, while).
- Automatiser les tâches d'administration sur les fichiers et les comptes utilisateurs (tests, parcours de dossiers, lecture de fichiers ligne par ligne).
- Surveiller et piloter les processus depuis un script : PID, tâches de fond, signaux et codes de retour.
- Construire des expressions régulières correctes et traiter du texte avec grep, sed et awk (champs, filtres, calculs).
- Planifier et automatiser des sauvegardes horodatées avec tar et cron, avec rotation des anciennes archives.
- Sécuriser ses scripts (set -euo pipefail, guillemets, validation des paramètres) et journaliser chaque action importante.
- Assembler une boîte à outils d'administration modulaire, documentée et testée.

Leçon 1 — Le shell : variables, conditions et boucles

Un script bash commence presque toujours par la ligne #!/bin/bash, appelée shebang : elle indique au système quel interpréteur doit exécuter le fichier quand on le lance directement. Viennent ensuite les variables, créées par affectation, par exemple nom="Awa". Attention à la règle la plus violée par les débutants : aucun espace autour du signe =. Écrire nom = "Awa" lance en réalité la commande nom avec des arguments, et échoue. Pour lire une variable, on écrit $nom ou ${nom} ; pour stocker le résultat d'une commande, on utilise la substitution : lignes=$(wc -l < fichier). Les calculs entiers se font dans $((...)), par exemple somme=$((somme + i)). Enfin, le script reçoit des paramètres : $1, $2 sont les arguments, $# leur nombre, $0 le nom du script et $? le code de retour de la dernière commande (0 = succès).

La condition if s'appuie sur la commande test, presque toujours écrite entre crochets : if [condition]; then ... else ... fi. Les crochets exigent des espaces autour d'eux et autour des opérateurs. On compare les nombres avec -eq, -ne, -lt, -le, -gt, -ge, les chaînes avec = et !=, et on teste les fichiers avec -e (existe), -f (fichier ordinaire), -d (dossier), -s (non vide), -r et -x (lisible, exécutable). Pour les choix multiples sur une

même valeur, la structure case est plus lisible qu'une cascade de if : case "$choix" in a) ... ;; *) ... ;; esac. Entoure toujours les variables de guillemets dans les tests : sans eux, une variable vide ou contenant un espace rend la condition fausse ou provoque une erreur de syntaxe incompréhensible.

Bash offre trois boucles. La boucle for parcourt une liste : for i in 1 2 3, for f in *.txt, ou la forme arithmétique for ((i=0; i<5; i++)). La boucle while répète tant qu'une condition est vraie : indispensable pour lire un fichier ligne par ligne avec while IFS= read -r ligne; do ...; done < fichier, ou pour attendre un événement. La boucle until est la négation de while. Deux mots-clés contrôlent le déroulement : break sort de la boucle, continue passe directement au tour suivant. Comme en algorithmique, la question à se poser avant d'écrire une boucle reste la même : qu'est-ce qui change à chaque tour, et pourquoi cela finira-t-il ? Un compteur qui n'avance jamais produit une boucle infinie, qu'on interrompt par Ctrl+C.

L'essentiel
Affectation sans espaces autour de =, lecture par $variable, calculs dans $(( )), tests entre [ ] avec les opérateurs -eq/-lt/-f..., boucles for, while et until : c'est la grammaire minimale de tout script bash.

Exemple — Premier script complet : variables, if, for et while

```
#!/bin/bash
# l1.sh : variables, condition et boucles
utilisateur="Awa"
seuil=10
note=14

echo "Bonjour $utilisateur"
if [ "$note" -ge "$seuil" ]; then
echo "Note $note : admis"
else
echo "Note $note : ajourne"
fi

somme=0
for i in 1 2 3 4 5; do
somme=$((somme + i))
done
echo "Somme de 1 a 5 : $somme"

compteur=3
while [ "$compteur" -gt 0 ]; do
echo "Plus que $compteur..."
compteur=$((compteur - 1))
done
echo "Go !"

# ---------- Sortie obtenue ----------
# Bonjour Awa
# Note 14 : admis
# Somme de 1 a 5 : 15
# Plus que 3...
# Plus que 2...
# Plus que 1...
# Go !
```

À retenir

• nom=valeur, sans aucun espace autour du = ; $nom pour lire, $(commande) pour capturer une sortie.

• Les tests [...] comparent les nombres (-ge, -lt), les chaînes (=, !=) et les fichiers (-f, -d, -s).

• for pour parcourir une liste connue, while pour répéter sous condition ou lire un fichier ligne par ligne.

Exercice

Écris un script moyenne.sh qui reçoit une note en paramètre ($1). Si aucun paramètre n'est fourni (teste $#), le script affiche un message d'usage et se termine avec le code 2. Sinon, il affiche « Mention Très Bien » si la note est au moins 16, « Bien » si au moins 14, « Assez Bien » si au moins 12, « Passable » si au moins 10, et « Ajourné » en dessous. Teste le script avec les valeurs 17, 13,5 n'est pas accepté : les comparaisons [ ] de bash ne gèrent que les entiers — utilise 17, 13, 10 et 8.

✔ Corrigé
Script : #!/bin/bash ; if [$# -eq 0]; then echo "Usage : $0 NOTE" ; exit 2 ; fi ; note=$1 ; puis la cascade : if ["$note" -ge 16] → Très Bien ; elif ["$note" -ge 14] → Bien ; elif ["$note" -ge 12] → Assez Bien ; elif ["$note" -ge 10] → Passable ; else → Ajourné ; fi. Résultats obtenus aux tests : 17 → Très Bien ; 13 → Assez Bien (13 est inférieur à 14 mais supérieur ou égal à 12 : la cascade teste du plus exigeant au moins exigeant, et le premier test vrai l'emporte) ; 10 → Passable ; 8 → Ajourné. Sans paramètre, le script affiche l'usage et echo $? après l'appel donne 2, ce qui prouve que le code de sortie distingue bien une erreur d'utilisation d'un succès.

Leçon 2 — Scripts d'administration : fichiers et utilisateurs

Le premier terrain d'application des scripts est le système de fichiers. Un schéma revient constamment : parcourir un ensemble de fichiers avec for f in "$dossier"/*.txt, vérifier chaque élément avec un test (-f, -s), agir dessus (compter ses lignes avec wc -l, le copier, le compresser) et tenir un compteur. La substitution de commande transforme le résultat d'un outil en données du script : lignes=$(wc -l < "$fichier") — remarque la redirection < qui fait afficher le nombre seul, sans le nom du fichier. Autre schéma fondamental : lire un fichier de paramètres ligne par ligne avec while IFS= read -r nom; do ...; done < liste.txt. Le IFS= protège les espaces en début et fin de ligne, et l'option -r empêche read d'interpréter les barres obliques inverses : c'est la forme sûre, à utiliser telle quelle.

Côté utilisateurs, le script ne doit jamais créer un compte les yeux fermés. Avant un useradd, on vérifie que le compte n'existe pas déjà : la commande id nomUtilisateur réussit (code 0) si le compte existe, échoue sinon ; grep -q "^awa:" /etc/passwd fait le même travail par recherche du login en début de ligne du fichier des comptes. La création d'un lot de comptes suit alors le schéma de lecture de fichier : pour chaque ligne contenant un login et un nom complet, tester l'existence, créer si absent (useradd -m), sinon signaler « déjà présent », et compter les créations et les sauts. Les opérations qui modifient le système exigent les droits d'administration : le script doit soit être lancé avec sudo, soit vérifier qu'il est root avec [$(id -u) -eq 0] et refuser poliment de continuer sinon.

Un bon script d'administration rend compte de ce qu'il a fait. Compte les éléments traités, ceux modifiés, ceux ignorés, et affiche un bilan final : c'est ce qui distingue un script fiable d'un script dangereux. Il doit aussi être idempotent quand c'est possible : le relancer deux fois ne doit pas tout casser, mais produire le même état final — notre vérification « le compte existe déjà » sert exactement à cela. Enfin, teste toujours sur un dossier ou un lot réduit avant de lancer le traitement complet : en administration, le mode d'emploi commence par un essai sur copie, jamais sur les vraies données d'un service en production.

L'essentiel
Parcourir avec for et des motifs de fichiers, lire un fichier paramètre avec while IFS= read -r, tester l'existence avant de créer (id, -e), compter et afficher un bilan : le squelette de presque tous les scripts d'administration.

Exemple — Inventaire d'un dossier et vérification d'un compte

```
#!/bin/bash
# l2.sh : traiter tous les fichiers .txt d'un dossier
dossier="classe"

for fichier in "$dossier"/*.txt; do
if [ -f "$fichier" ]; then
lignes=$(wc -l < "$fichier")
echo "$fichier : $lignes ligne(s)"
fi
done

if id awa >/dev/null 2>&1; then
echo "Le compte awa existe"
else
echo "Le compte awa n'existe pas sur cette machine"
fi

compte=$(ls "$dossier" | wc -l)
echo "Total : $compte fichier(s) dans $dossier"

# ---------- Sortie obtenue ----------
# classe/awa.txt : 0 ligne(s)
# classe/fatou.txt : 0 ligne(s)
# classe/moussa.txt : 0 ligne(s)
# Le compte awa n'existe pas sur cette machine
# Total : 3 fichier(s) dans classe
```

À retenir

- while IFS= read -r variable; do …; done < fichier : la forme sûre pour traiter un fichier ligne par ligne.
- Toujours tester avant d'agir : id pour un compte, -e ou -f pour un chemin, puis compter et afficher le bilan.
- Un script relançable sans dommage (idempotent) vaut dix scripts qu'on n'ose plus exécuter.

Exercice

Écris un script dossiers.sh qui lit un fichier eleves.txt contenant un prénom par ligne (awa, moussa, fatou) et crée, pour chacun, un dossier dossiers/PRENOM s'il n'existe pas déjà, en affichant « Créé : dossiers/awa » ou « Déjà présent : dossiers/awa ». Relance le script une deuxième fois et explique ce qui doit se passer si le script est bien idempotent.

✔ Corrigé
Script : #!/bin/bash ; mkdir -p dossiers ; while IFS= read -r nom; do if [-d "dossiers/$nom"]; then echo "Deja present : dossiers/$nom" ; else mkdir "dossiers/$nom" ; echo "Cree : dossiers/$nom" ; fi ; done < eleves.txt. Première exécution testée : trois lignes « Cree : dossiers/awa », « Cree : dossiers/moussa », « Cree : dossiers/fatou », et ls dossiers affiche awa fatou moussa. Deuxième exécution : trois lignes « Deja present : ... » et aucun message d'erreur, car le test -d a évité de recréer ce qui existe : le script est idempotent, l'état final est identique après un ou dix passages.

Leçon 3 — Gestion des processus en ligne de commande

Chaque programme lancé devient un processus, identifié par un numéro unique, le PID. Un script connaît le sien dans la variable $$, ce qui sert à créer des fichiers temporaires qui ne se mélangeront pas entre deux exécutions simultanées : tmp="/tmp/rapport.$$.txt". Pour observer les processus, ps aux affiche la liste complète avec l'utilisateur, le PID, l'occupation processeur et mémoire, et la commande ; la forme ciblée ps -o pid,stat,cmd -p PID montre un seul processus et son état (STAT : R en cours d'exécution, S en attente — l'état normal d'un programme qui dort ou attend, T arrêté, Z zombie). Combiné à grep, ps répond à la question quotidienne : « est-ce que tel service tourne ? » — par exemple ps aux | grep -c "[a]pache2" (les crochets autour de la première lettre évitent que grep se compte lui-même dans le résultat).

Un script peut lancer des tâches en arrière-plan avec & : traitement & puis travail=$! — la variable $! contient le PID du dernier processus lancé en fond. On suit son état avec jobs, on l'attend avec wait PID (qui rend la main seulement quand la tâche se termine, et rend son code de retour), et on dialogue avec lui par des signaux envoyés par kill : SIGTERM (15, le défaut) demande un arrêt propre, SIGKILL (9) tue sans discussion — à n'utiliser qu'en dernier recours car le programme ne peut ni sauvegarder ni nettoyer — et SIGHUP (1) signifie classiquement « raccrochage », souvent interprété comme « relis ta configuration ». Avant de tuer, on vérifie toujours que le PID est bien celui du bon processus : un kill sur un PID recyclé peut arrêter un service innocent.

Le code de retour, stocké dans $?, est le langage par lequel les processus rendent compte : 0 succès, autre chose échec (1 erreur générale, 2 mauvaise utilisation, 127 commande introuvable). Un script bien écrit teste les codes de retour de ses commandes sensibles ou enchaîne avec les opérateurs && (n'exécuter la suite que si la commande précédente a réussi) et || (n'exécuter que si elle a échoué). C'est ce qui permet des lignes expressives comme : cp fichier /backup/ && echo "copie OK" || echo "ECHEC de copie". Retiens la logique : en shell, « vrai » signifie code 0, et c'est ce code, pas un texte affiché, que if, && et while évaluent réellement.

L'essentiel
$$ = PID du script, $! = PID de la dernière tâche de fond, $? = code de retour (0 = vrai/succès). ps pour observer, kill pour envoyer un signal (TERM d'abord, KILL en dernier recours), wait pour attendre une tâche de fond.

Exemple — Lancer, observer et arrêter proprement une tâche de fond

```
#!/bin/bash
# l3.sh : surveiller les processus
echo "--- Mon shell et son PID ---"
echo "PID du script : $$"
sleep 30 &
pid_fond=$!
echo "Processus en tache de fond : $pid_fond"
ps -o pid,stat,cmd -p "$pid_fond"
kill "$pid_fond"
wait "$pid_fond" 2>/dev/null
echo "Processus $pid_fond arrete proprement"

# ---------- Sortie obtenue ----------
# --- Mon shell et son PID ---
# PID du script : 3016
# Processus en tache de fond : 3024
# PID STAT CMD
# 3024 S sleep 30
# Processus 3024 arrete proprement
```

À retenir

- ps -o pid,stat,cmd -p PID décrit un processus précis ; STAT S = en attente, R = en cours, Z = zombie.
- kill envoie SIGTERM par défaut (arrêt propre demandé) ; kill -9 force, sans laisser au programme le temps de nettoyer.
- wait PID attend la fin d'une tâche de fond ; le code dans $? dit si tout s'est bien passé.

Exercice

1) Dans un terminal, lance sleep 120 & puis affiche la liste de tes tâches avec jobs et le détail du processus avec ps. 2) Arrête-le avec kill et le bon PID, puis vérifie avec ps -p que le processus a disparu (ps n'affiche plus que l'en-tête). 3) Explique pourquoi, dans un script, il est dangereux d'écrire kill suivi d'un numéro de PID recopié à la main depuis un ancien affichage.

✔ Corrigé
1) jobs affiche « [1]+ Running sleep 120 & » ; ps -o pid,stat,cmd -p $! montre le sleep à l'état S. 2) kill $! envoie SIGTERM : sleep se termine immédiatement ; wait $! se débloque ; un nouveau ps -p sur ce PID n'affiche plus aucune ligne de processus, seulement l'en-tête PID STAT CMD. 3) Les PID sont recyclés par le système : quand un processus meurt, son numéro redevient disponible pour un programme lancé plus tard. Un numéro recopié d'un ancien affichage peut donc désigner, quelques minutes après, un tout autre programme — potentiellement un service important. La bonne pratique est de capturer le PID au moment du lancement ($!) ou de le relire juste avant l'action, et de vérifier la commande associée (colonne CMD de ps) avant d'envoyer le signal.

Leçon 4 — Expressions régulières et traitement de texte avec grep, sed et awk

Une expression régulière (ou regex) décrit un motif de texte plutôt qu'un texte exact. Les briques de base : . remplace n'importe quel caractère, * signifie « le caractère précédent, zéro ou plusieurs fois », + (en mode étendu) « une ou plusieurs fois », ? « zéro ou une fois », [0-9] un caractère parmi un intervalle, [^0-9] tout caractère sauf un chiffre, ^ le début de ligne et $ la fin de ligne. Ainsi ^[0-9]{8}$ décrit exactement huit chiffres et rien d'autre — le format d'un numéro de téléphone malien sans indicatif. grep utilise par défaut les regex de base, où +, ? et { } doivent être échappés pour jouer leur rôle spécial ; avec l'option -E (regex étendues), ils s'écrivent directement. Ce détail explique le classique « ma regex ne trouve rien » : le motif était écrit pour -E mais lancé sans.

Les trois grands outils se partagent le travail. grep filtre les lignes qui correspondent (ou non, avec -v) à un motif, les compte avec -c, les numérote avec -n. sed transforme le texte au vol : sa commande s/ancien/nouveau/ remplace la première occurrence de chaque ligne, s/ancien/nouveau/g toutes les occurrences, et sed -i modifie le fichier sur place — puissant, donc à tester d'abord sans -i. awk, enfin, découpe chaque ligne en champs $1, $2, … selon un séparateur (-F';'), applique des conditions ($2 >= 10), affiche, calcule des sommes et des moyennes dans son bloc END. Règle de choix : filtrer → grep ; remplacer → sed ; colonnes et calculs → awk. Et les trois se chaînent par des tubes avec sort, uniq et wc pour produire de vrais rapports.

Le traitement de texte est le super-pouvoir de l'administrateur, car les journaux système et applicatifs sont du texte. Le schéma d'analyse d'un journal d'accès web est toujours le même : extraire un champ avec awk, trier, compter les occurrences avec uniq -c, trier par effectif décroissant. Quelques pièges à connaître : les guillemets simples protègent le motif de l'interprétation du shell (indispensable autour des programmes awk et des motifs contenant $) ; le séparateur de champs par défaut d'awk est l'espace, donc un fichier en point-virgules exige -F';' ; et une regex trop gourmande comme .* peut avaler plus que prévu. Teste chaque motif sur un petit fichier dont tu connais la réponse avant de l'appliquer au journal complet.

L'essentiel
Regex : . * + ? [ ] ^ $ décrivent des motifs. grep -E pour filtrer avec les regex étendues, sed s/a/b/g pour remplacer, awk -F et $1, $2 pour travailler par colonnes et calculer.

Exemple — Filtrer, découper, calculer et remplacer avec grep, awk et sed

```
#!/bin/bash
# l4.sh : grep, sed, awk sur notes.txt et acces.log
echo "--- Etudiants de Bamako ---"
grep "Bamako" notes.txt
echo "--- Notes >= 10 (awk) ---"
awk -F';' '$2 >= 10 {print $1, ":", $2}' notes.txt
echo "--- Moyenne des notes ---"
awk -F';' '{s += $2} END {print s / NR}' notes.txt
echo "--- Codes HTTP 4xx dans acces.log ---"
grep -E '" 4[0-9][0-9] ' acces.log
echo "--- Remplacement Segou -> Sikasso (sed) ---"
sed 's/Segou/Sikasso/' notes.txt

# Fichier notes.txt :
# Awa Diallo;14;Bamako / Moussa Traore;9;Segou
# Fatou Keita;16;Bamako / Ibrahim Sissoko;11;Kayes
# Aminata Diarra;7;Bamako
#
# ---------- Sortie obtenue ----------
# --- Etudiants de Bamako ---
# Awa Diallo;14;Bamako
# Fatou Keita;16;Bamako
# Aminata Diarra;7;Bamako
# --- Notes >= 10 (awk) ---
# Awa Diallo : 14
# Fatou Keita : 16
# Ibrahim Sissoko : 11
# --- Moyenne des notes ---
# 11.4
# --- Codes HTTP 4xx dans acces.log ---
# 192.168.1.10 - - [11/Oct/2026:08:15:02] "POST /login" 403 128
# 192.168.1.13 - - [11/Oct/2026:08:17:20] "GET /photo.png" 404 64
# --- Remplacement Segou -> Sikasso (sed) ---
# Awa Diallo;14;Bamako
# Moussa Traore;9;Sikasso
# Fatou Keita;16;Bamako
# Ibrahim Sissoko;11;Kayes
# Aminata Diarra;7;Bamako
```

À retenir

• grep -E active les regex étendues (+, ?, {n} sans barre oblique) ; ^ et $ ancrent le motif en début et fin de ligne.

• sed 's/a/b/' remplace une fois par ligne, 's/a/b/g' partout ; -i écrit dans le fichier — à tester sans -i d'abord.

• awk -F';' découpe en champs $1, $2… ; le bloc END sert aux bilans (sommes, moyennes avec NR).

Exercice

Dans un fichier numeros.txt contenant une valeur par ligne (70112233, 123, 0022370112233, 90112233, abc), écris la commande grep qui n'affiche que les numéros maliens valides formés d'exactement 8 chiffres. Explique le rôle de chaque élément du motif, puis dis pourquoi la ligne 0022370112233 (indicatif + numéro collés) est rejetée.

✔ Corrigé
Commande testée : grep -E '^[0-9]{8}$' numeros.txt — elle affiche exactement deux lignes : 70112233 et 90112233. Décorticage : ^ impose le début de la ligne (rien avant) ; [0-9] accepte un caractère chiffre ; {8} répète exactement huit fois le motif précédent ; $ impose la fin de ligne (rien après). La ligne 0022370112233 contient 13 chiffres : après les 8 premiers, il reste des caractères avant la fin de ligne, donc le motif ancré ne correspond pas — c'est précisément le rôle des ancres ^ et $: sans elles, grep aurait trouvé « 8 chiffres quelque part » dans presque toutes les lignes, y compris dans 123… non, 123 n'en a que trois, mais 0022370112233 aurait été accepté à tort.

Leçon 5 — Planification et automatisation des sauvegardes

Une sauvegarde manuelle est une sauvegarde qu'on oubliera. La sauvegarde professionnelle repose sur trois pièces : un script qui produit une archive horodatée, un planificateur qui le lance tout seul, et une rotation qui supprime les archives trop anciennes pour ne pas remplir le disque. L'archivage se fait avec tar : tar -czf archive.tar.gz dossier crée une archive compressée (c créer, z compresser en gzip, f fichier), tar -tzf la liste sans l'extraire, tar -xzf la restaure. L'horodatage vient de la commande date : date +%Y%m%d-%H%M%S produit par exemple 20261011-163428, et ce format année-mois-jour a une vertu précieuse : trié par ordre alphabétique, il donne l'ordre chronologique. Construire le nom par variable — archive="$dest/sauvegarde-$date_jour.tar.gz" — rend le script lisible et réutilisable.

Le planificateur cron lit la crontab de chaque utilisateur, éditée par crontab -e et listée par crontab -l. Chaque ligne contient cinq champs de temps — minute, heure, jour du mois, mois, jour de semaine (0 ou 7 = dimanche) — suivis de la commande. Ainsi, 0 2 * * * /home/adminiam/scripts/sauvegarde.sh lance le script chaque nuit à 2 h 00, et 30 3 * * 1 le lance le lundi à 3 h 30. Deux pièges classiques : cron ignore l'environnement interactif (pas d'alias, PATH minimal), donc les scripts planifiés utilisent des chemins absolus ; et cron n'affiche rien, donc le script doit écrire son bilan dans un journal et, en cas d'échec, laisser une trace visible. Une erreur fréquente consiste aussi à confondre les champs : 0 2 * * 0 ne signifie pas « toutes les 2 heures » mais « le dimanche à 2 h ».

La rotation complète le dispositif : find "$dest" -name "sauvegarde-*.tar.gz" -mtime +7 supprime (avec -delete, après un premier essai sans) les archives de plus de sept jours. Avant de faire confiance au système, on le prouve : lancer le script à la main,

vérifier l'archive créée (tar -tzf doit lister les fichiers attendus), simuler une restauration dans un dossier temporaire, puis seulement installer la ligne cron et vérifier le lendemain que l'exécution nocturne a bien eu lieu en lisant le journal. Une sauvegarde jamais restaurée en test est un espoir, pas une sauvegarde. Retiens la règle de ce cours : non testée = non faite.

L'essentiel
tar -czf crée l'archive horodatée (date +%Y%m%d-%H%M%S), cron la déclenche (5 champs : minute heure jour mois jour-semaine), find -mtime fait la rotation. On ne fait confiance qu'à une sauvegarde dont la restauration a été testée.

Exemple — Script de sauvegarde horodatée avec vérification du contenu

```
#!/bin/bash
# sauvegarde.sh : archive horodatee d'un dossier
source_dir="classe"
dest_dir="sauvegardes"
date_jour=$(date +%Y%m%d-%H%M%S)
archive="$dest_dir/sauvegarde-$date_jour.tar.gz"

mkdir -p "$dest_dir"
tar -czf "$archive" "$source_dir"
echo "Archive creee : $archive"
ls -lh "$archive"
echo "--- Contenu ---"
tar -tzf "$archive"

# Ligne crontab associee (crontab -e) :
# 0 2 * * * /home/adminiam/scripts/sauvegarde.sh
#
# ---------- Sortie obtenue ----------
# Archive creee : sauvegardes/sauvegarde-20261011-163428.tar.gz
# -rw-rw---- 1 root root 175 Oct 11 16:34
sauvegardes/sauvegarde-20261011-163428.tar.gz
# --- Contenu ---
# classe/
# classe/fatou.txt
# classe/moussa.txt
# classe/awa.txt
```

À retenir

- tar -czf / -tzf / -xzf : créer, lister, extraire. Vérifier le contenu d'une archive fait partie de la sauvegarde.
- Une ligne cron = 5 champs de temps + commande en chemin absolu ; 0 2 * * * signifie chaque jour à 2 h 00.
- Rotation avec find -mtime +N, testée d'abord sans -delete ; restauration testée avant de déclarer le système fiable.

Exercice

1) Écris la ligne crontab qui lance /home/adminiam/scripts/sauvegarde.sh tous les jours à 23 h 45, et celle qui le lance uniquement le dimanche à 4 h 15. 2) Écris la commande find qui liste les archives sauvegarde-*.tar.gz du dossier sauvegardes vieilles de plus de 30 jours, sans les supprimer. 3) Explique pourquoi on commence toujours par lister avant de supprimer.

✔ Corrigé
1) Tous les jours à 23 h 45 : « 45 23 * * * /home/adminiam/scripts/sauvegarde.sh ». Le dimanche à 4 h 15 : « 15 4 * * 0 /home/adminiam/scripts/sauvegarde.sh » (le 0 du cinquième champ = dimanche ; 7 est aussi accepté). 2) find sauvegardes -name "sauvegarde-*.tar.gz" -mtime +30 — testée dans ce cours avec un seuil de 7 jours sur un fichier daté artificiellement de 10 jours : elle a listé exactement ce fichier et ignoré le fichier récent. 3) -delete supprime sans confirmation et sans corbeille : une erreur de motif (par exemple un nom de dossier trop large) détruirait des fichiers sains. Lister d'abord, regarder la liste, compter, puis seulement ajouter -delete : c'est la même prudence que pour sed -i ou kill.

Leçon 6 — Sécuriser ses scripts et journaliser les actions

Un script d'administration concentre les pouvoirs de root sur des données réelles : sa robustesse n'est pas un luxe. La première protection tient en une ligne placée juste après le shebang : set -euo pipefail. -e arrête le script dès qu'une commande échoue (au lieu de continuer sur des données incohérentes), -u transforme l'utilisation d'une variable non définie en erreur immédiate — c'est lui qui révèle les fautes de frappe dans les noms de variables — et -o pipefail fait échouer tout un tube dès qu'une de ses commandes échoue, au lieu de ne retenir que le code de la dernière. Ces options ont attrapé une vraie erreur pendant la préparation de ce cours : un $USER non défini dans l'environnement de test a été signalé immédiatement au lieu de produire un journal silencieusement faux. Attention : -e a des angles morts (conditions de if, commandes dans un tube suivi de || true), donc on continue à tester explicitement les opérations critiques.

Les autres règles de sécurité sont tout aussi concrètes. Entoure systématiquement les variables de guillemets ("$fichier") : sans eux, un nom contenant un espace ou un caractère * est découpé ou développé par le shell, avec des conséquences allant de l'erreur bénigne à la suppression du mauvais fichier. Valide les paramètres dès l'entrée : tester $#, vérifier qu'un dossier existe avant d'y écrire, refuser les valeurs vides avec ${1:?message d'erreur} ou une valeur par défaut ${1:-defaut}. Ne construis jamais une commande en collant une saisie utilisateur dans un eval. Garde les scripts d'administration en lecture-écriture pour root seul (chmod 700) : un script modifiable par tous et exécuté par cron avec les droits root est une porte d'entrée offerte. Enfin, shellcheck, quand il est disponible, relit le script et signale la plupart de ces fautes avant l'exécution.

La journalisation (logging) est la mémoire du script. Chaque action importante — démarrage, création, suppression, erreur, fin — est ajoutée à un fichier journal avec un horodatage : echo "$(date '+%Y-%m-%d %H:%M:%S') - message" >> journal.log. Le

>> ajoute sans écraser ; écrire dans /var/log exige les droits root, donc les scripts d'utilisateur journaliseront dans leur dossier. Définir une petite fonction journaliser() évite de répéter le format de date et garantit des lignes homogènes, faciles à relire avec grep (par exemple grep "ERREUR" journal.log). Un bon journal répond, des jours plus tard, aux questions : quand le script a-t-il tourné, avec quels paramètres, qu'a-t-il fait, et où s'est-il arrêté ? C'est lui qui transforme un « ça ne marche plus » en diagnostic.

L'essentiel
set -euo pipefail en première ligne, des guillemets autour de chaque variable, des paramètres validés, des droits 700 sur les scripts d'administration, et une fonction de journalisation horodatée : le minimum vital d'un script de production.

Exemple — Script sécurisé (set -euo pipefail) avec fonction de journalisation

```
#!/bin/bash
# l6.sh : script securise et journalise
set -euo pipefail

journal="actions.log"

journaliser() {
echo "$(date '+%Y-%m-%d %H:%M:%S') - $1" >> "$journal"
}

journaliser "Debut du script par $(whoami)"
nom=${1:-"inconnu"}
journaliser "Parametre recu : $nom"
journaliser "Fin du script"
echo "Actions journalisees dans $journal :"
cat "$journal"

# ---------- Sortie obtenue (appel : bash l6.sh rapport) ----------
# Actions journalisees dans actions.log :
# 2026-10-11 16:34:44 - Debut du script par root
# 2026-10-11 16:34:44 - Parametre recu : rapport
# 2026-10-11 16:34:44 - Fin du script
```

À retenir

- set -euo pipefail : arrêt à la première erreur, erreur sur variable non définie, échec des tubes détecté.
- Guillemets partout, paramètres contrôlés (${1:-defaut}), scripts sensibles en chmod 700, jamais d'eval sur une saisie.
- Journaliser = écrire, avec date et heure, chaque action importante dans un fichier dédié, via une fonction unique.

Exercice

Voici un script fragile : #!/bin/bash ; fichier=$1 ; cp $fichier /var/sauvegardes/ ; rm $fichier. Cite au moins quatre problèmes précis et réécris le script en version sûre :

vérification du nombre de paramètres, test d'existence du fichier, guillemets, set -euo pipefail, journalisation de la copie, et suppression seulement si la copie a réussi.

✔ Corrigé
Problèmes : 1) $1 n'est jamais vérifié — sans paramètre, les commandes travaillent sur du vide ; 2) $fichier sans guillemets casse sur les noms avec espaces et peut développer un motif * ; 3) le rm s'exécute même si le cp a échoué : on peut perdre l'unique exemplaire du fichier ; 4) aucune trace écrite de l'opération ; 5) aucun set -e : les erreurs défilent et le script se termine « normalement ». Version sûre : #!/bin/bash ; set -euo pipefail ; if [$# -ne 1]; then echo "Usage : $0 FICHIER" >&2 ; exit 2 ; fi ; fichier=$1 ; if [! -f "$fichier"]; then echo "Introuvable : $fichier" >&2 ; exit 1 ; fi ; cp "$fichier" /var/sauvegardes/ && echo "$(date '+%F %T') - copie de $fichier OK" >> copie.log ; rm "$fichier". Grâce à set -e et à l'enchaînement, si la copie échoue le script s'arrête avant le rm : le fichier source survit, et le journal ne contient que des copies réussies.

Leçon 7 — Mini-projet : boîte à outils d'administrateur système

Le mini-projet assemble tout le cours en un seul livrable : un script unique, boite.sh, qui centralise les petites vérifications qu'un administrateur fait dix fois par jour, et qui répond de façon structurée grâce à case. Les fonctions découpent le travail : une fonction usage() qui explique l'emploi du script, puis une entrée de menu par action — disque (df), mémoire (free), espace des dossiers (du), utilisateurs connectés (who), état d'un service (systemctl is-active), dernières lignes d'un journal. Chaque action est courte, testée séparément avant d'être intégrée, et produit une ligne de synthèse lisible plutôt qu'un écran brut : c'est awk qui extrait les champs utiles de df et de free pour composer ces phrases. L'ensemble hérite des bonnes pratiques du cours : set -euo pipefail, paramètres validés par le case (tout choix inconnu affiche l'usage et termine avec le code 1), et journalisation des appels.

La méthode de construction est aussi importante que le résultat. On commence par un cahier des charges d'une page : quelles actions, quelles entrées, quelles sorties, quels codes de retour. On développe ensuite action par action, en testant chacune immédiatement — jamais deux fonctions nouvelles sans test entre les deux — puis on écrit les tests de bout en bout : appel sans paramètre, appel avec un choix inconnu, appel normal pour chaque action. Les résultats obtenus pendant la préparation de cette leçon illustrent la démarche : les trois actions disque, mémoire et qui ont répondu correctement, et l'appel avec le choix « bidon » a affiché l'usage et rendu le code 1, exactement comme spécifié. Les valeurs affichées (pourcentages, mémoire) dépendent évidemment de la machine qui exécute le script.

Une boîte à outils vit et grandit : chaque nouvelle tâche récurrente devient une entrée de menu. Pour la faire durer, on la documente : un en-tête de commentaires (rôle, auteur, date, exemples d'appel), un usage() toujours à jour, et un fichier LISEZ-MOI dans le dossier des scripts. On la versionne comme du code (dépôt Git local suffit) afin de pouvoir revenir en arrière. Et on la fait relire : un collègue qui utilise la boîte sans explication orale est le meilleur test d'ergonomie. Le TP qui suit ce cours te demande

d'en construire ta propre version complète, évaluée sur le fonctionnement, la sécurité et la documentation — exactement les critères d'un outil professionnel.

L'essentiel
La boîte à outils = un script modulaire (fonctions + case), des sorties de synthèse composées avec awk, les protections du cours (set -euo pipefail, validation, codes de retour) et une documentation tenue à jour au fil des ajouts.

Exemple — Boîte à outils minimale : trois actions testées et un usage propre

```
#!/bin/bash
# boite.sh : mini boite a outils d'administration
set -euo pipefail

usage() {
echo "Usage : $0 {disque|memoire|qui}"
}

case "${1:-}" in
disque)
df -h . | tail -1 | awk '{print "Disque : " $5 " utilise (" $3 "/" $2 ")"}'
;;
memoire)
free -m | awk 'NR==2 {print "Memoire : " $3 " Mo utilises sur " $2 " Mo"}'
;;
qui)
echo "Connecte : $(whoami) sur $(hostname)"
;;
*)
usage
exit 1
;;
esac

# ---------- Sorties obtenues ----------
# $ bash boite.sh disque
# Disque : 1% utilise (896K/512M)
# $ bash boite.sh memoire
# Memoire : 6994 Mo utilises sur 7936 Mo
# $ bash boite.sh qui
# Connecte : root sur htch-runtime
# $ bash boite.sh bidon
# Usage : boite.sh {disque|memoire|qui} (code de retour : 1)
```

À retenir

- Découper en fonctions, aiguiller avec case, faire échouer proprement (usage + code 1) sur tout paramètre inconnu.
- Construire et tester action par action ; les tests d'appel (sans paramètre, paramètre faux, appels normaux) font partie du livrable.
- Documenter en tête de script et faire évoluer l'outil au fil des tâches récurrentes, sous gestion de versions.

Exercice

Ajoute deux actions à la boîte : « dossiers » qui affiche les trois plus gros dossiers du répertoire courant (du -sh */ 2>/dev/null, tri décroissant, trois premiers), et « service NOM » qui affiche si le service passé en deuxième paramètre est actif (systemctl is-active) ou rapporte proprement qu'il ne l'est pas. Pense au cas où le deuxième paramètre manque.

✔ Corrigé
Action dossiers : du -sh */ 2>/dev/null | sort -rh | head -3 — 2>/dev/null masque les erreurs de permission sur les dossiersillisibles, sort -rh trie en lisible décroissant, head -3 limite l'affichage. Action service : service=${2:-""} ; if [-z "$service"]; then echo "Usage : $0 service NOM" ; exit 2 ; fi ; if systemctl is-active --quiet "$service"; then echo "Service $service : actif" ; else echo "Service $service : non actif ou inconnu" ; fi. Le test --quiet exploite le code de retour de systemctl sans analyser de texte, dans l'esprit du cours. Ne pas oublier de compléter usage() avec les deux nouvelles actions : une documentation qui ment est pire qu'une documentation absente.

Travaux dirigés (TD)

TD 1 — Analyse automatisée d'un journal d'accès web

Le fichier acces.log contient 6 lignes au format simplifié : IP - - [date] "METHODE page" CODE TAILLE, par exemple : 192.168.1.10 - - [11/Oct/2026:08:12:03] "GET /index.html" 200 512. On y trouve aussi un POST /login en 403, un GET /photo.png en 404 et trois autres requêtes en 200. Écris un script analyse.sh qui affiche : 1) le nombre total de requêtes (wc) ; 2) le nombre de requêtes par code HTTP (awk + sort + uniq -c) — attention, dans ce format simplifié le code est le 7e champ, pas le 9e comme dans le format Apache complet ; 3) le nombre de requêtes par adresse IP, triées par effectif décroissant ; 4) les seules lignes en erreur 4xx, extraites par grep -E.

✔ Corrigé du TD 1
Script : #!/bin/bash ; set -euo pipefail ; journal=${1:-acces.log} ; echo "Total : $(wc -l < "$journal") requetes" ; echo "--- Par code HTTP ---" ; awk '{print $7}' "$journal" | sort | uniq -c | sort -rn ; echo "--- Par adresse IP ---" ; awk '{print $1}' "$journal" | sort | uniq -c | sort -rn ; echo "--- Erreurs 4xx ---" ; grep -E '" 4[0-9][0-9] ' "$journal". Résultats obtenus à l'exécution sur le journal de l'énoncé : total 6 requêtes ; codes : 4 fois 200, 1 fois 404, 1 fois 403 ; adresses : 2 requêtes pour 192.168.1.10, 2 pour 192.168.1.11, 1 pour 192.168.1.12, 1 pour 192.168.1.13 ; erreurs 4xx : la ligne du POST /login en 403 depuis 192.168.1.10 et la ligne du GET /photo.png en 404 depuis 192.168.1.13. Points de méthode : uniq ne compte que les lignes identiques consécutives, d'où le sort préalable obligatoire ; le motif grep '" 4[0-9][0-9] ' cible le code en position sûre grâce au guillemet fermant de la requête et à l'espace qui suit le code.

TD 2 — Préparation des dossiers d'une promotion

Le secrétariat fournit un fichier promotion.txt avec un login par ligne (par exemple adiallo, mtraore, fkeita, isissoko, adiarra). Écris un script preparation.sh qui, pour chaque login : 1) crée le dossier travaux/LOGIN s'il n'existe pas ; 2) y dépose un fichier

bienvenue.txt contenant « Bienvenue à LOGIN » (avec le vrai login) seulement si le dossier vient d'être créé ; 3) affiche une ligne de bilan par étudiant (Créé ou Déjà présent) puis un total. Le script doit pouvoir être relancé sans rien dupliquer ni écraser le travail des étudiants déjà installés. Le mot de passe et la création réelle des comptes (useradd) sont hors périmètre de ce TD : on ne prépare que les dossiers.

✔ Corrigé du TD 2
Script : #!/bin/bash ; set -euo pipefail ; base="travaux" ; mkdir -p "$base" ; crees=0 ; presents=0 ; while IFS= read -r login; do if [-d "$base/$login"]; then echo "Deja present : $login" ; presents=$((presents + 1)) ; else mkdir "$base/$login" ; echo "Bienvenue a $login" > "$base/$login/bienvenue.txt" ; echo "Cree : $login" ; crees=$((crees + 1)) ; fi ; done < promotion.txt ; echo "Bilan : $crees cree(s), $presents deja present(s)". Vérification effectuée sur le même principe pendant la préparation du cours (trois logins) : la première exécution crée les trois dossiers avec leur fichier de bienvenue, la seconde n'affiche que des « Deja present » et un bilan 0 créé / 3 présents, sans modifier les fichiers existants — l'idempotence est démontrée par la double exécution, pas seulement affirmée. Le test -d est la clé : il transforme une opération destructive potentielle en simple constat.

Travaux pratiques (TP)

TP 1 — Boîte à outils complète et sauvegarde planifiée de srv-iam

Sur ta machine virtuelle, construis le livrable complet du cours dans un dossier scripts/ : 1) sauvegarde.sh — archive horodatée du dossier documents/ dans archives/, journalisation de chaque étape dans archives/sauvegarde.log, rotation des archives de plus de 7 jours (find -mtime), et arrêt propre en cas d'erreur ; 2) la ligne crontab qui lance cette sauvegarde chaque nuit à 2 h 30 ; 3) boite.sh enrichie avec les actions disque, memoire, qui, dossiers et service NOM, avec usage() complet. Fournis les preuves de test : sortie d'une exécution manuelle de la sauvegarde, contenu de l'archive (tar -tzf), restauration d'essai dans /tmp/restauration, journal produit, et sorties des cinq actions de la boîte (y compris un appel sans paramètre et un appel invalide).

✔ Attendus du TP 1
Attendus et barème (20 points). Sauvegarde (8 pts) : le script porte set -euo pipefail, construit le nom par date +%Y%m%d-%H%M%S (2 pts) ; tar -czf réussit et tar -tzf liste bien les fichiers de documents/ (2 pts) ; le journal contient les lignes horodatées de début, de création et de fin (2 pts) ; la rotation liste avant de supprimer et ne touche pas aux archives récentes (2 pts). Planification (3 pts) : la ligne « 30 2 * * * » suivie du chemin absolu du script figure dans crontab -l (2 pts) ; l'étudiant explique le rôle des cinq champs et le piège du PATH réduit de cron (1 pt). Boîte à outils (6 pts) : les cinq actions répondent juste sur la machine de test (3 pts) ; usage() est à jour et les appels invalides rendent un code non nul (2 pts) ; les sorties sont des synthèses d'une ligne, pas des écrans bruts (1 pt). Preuves (3 pts) : restauration d'essai réussie dans /tmp/restauration avec comparaison des fichiers (2 pts) ; dossier de tests complet et lisible (1 pt). Erreurs qui coûtent cher : suppression lancée sans test préalable, variable sans guillemets sur un chemin, sauvegarde déclarée bonne sans restauration testée.

Exercices corrigés supplémentaires

Exercice 1

Nettoyage des vieilles archives : écris la commande find qui liste les fichiers .tar.gz du dossier sauvegardes modifiés il y a plus de 7 jours, puis explique ce qu'il faut changer pour supprimer, et pourquoi on ne commence jamais par la version qui supprime.

✔ Corrigé
Liste : find sauvegardes -name "*.tar.gz" -mtime +7. Pendant la préparation du cours, cette commande, appliquée à un dossier contenant un fichier daté de 10 jours et un fichier récent, a listé exactement le fichier ancien et ignoré le récent. Pour supprimer, on ajoute -delete à la fin (ou -exec rm {} +). On commence par lister parce que find agit de façon récursive et silencieuse : un mauvais point de départ (par exemple . au lieu de sauvegardes) ou un motif trop large détruirait des fichiers sains, sans confirmation ni corbeille. La discipline est toujours : lister, regarder, compter, puis seulement supprimer.

Exercice 2

Débogage : le script suivant contient trois erreurs classiques. Trouve-les et corrige-les : #!/bin/bash ; dossier = "documents" ; if [-d $dossier]; then echo le dossier $dossier existe ; fi. Indice : les erreurs portent sur l'affectation, les crochets du test et les guillemets.

✔ Corrigé
Erreur 1 : « dossier = "documents" » met des espaces autour de = : bash cherche une commande nommée dossier et échoue. Forme juste : dossier="documents". Erreur 2 : « [-d $dossier] » colle les crochets au contenu : [est une commande qui exige un espace après [et avant]. Forme juste : [-d "$dossier"]. Erreur 3 : $dossier sans guillemets dans le test et dans le echo : si le nom contient un espace, le test reçoit deux arguments et plante. Script corrigé : #!/bin/bash ; dossier="documents" ; if [-d "$dossier"]; then echo "le dossier $dossier existe" ; fi. Ces trois fautes sont exactement celles que set -u, shellcheck et un premier test sur un cas simple révèlent en quelques secondes.

Exercice 3

Traitement par lot : écris un petit script qui, pour chaque fichier .log du dossier courant, affiche son nom et le nombre de lignes contenant le mot ERREUR (insensible à la casse). Que doit afficher le script pour un fichier sans aucune erreur ?

✔ Corrigé
Script : #!/bin/bash ; for f in *.log; do [-f "$f"] || continue ; n=$(grep -ci "erreur" "$f") ; echo "$f : $n ligne(s) d'erreur" ; done. grep -c compte les lignes correspondantes, -i rend la recherche insensible à la casse (ERREUR, erreur, Erreur sont comptés) et -ci combine les deux. Pour un fichier sans erreur, grep -c affiche 0 (le compteur fonctionne même sans correspondance) mais grep rend le code de retour 1 : c'est pourquoi le script n'utilise pas set -e ici, ou ajouterait || true après la substitution. Le test [-f "$f"] || continue protège le cas où aucun fichier .log n'existe : le motif *.log non développé resterait alors littéral et le compteur travaillerait sur un nom inexistant.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. La première ligne d'un script bash, #!/bin/bash, sert à…

   A) Définir le titre du script

   B) Indiquer l'interpréteur qui doit exécuter le fichier

   C) Rendre le fichier invisible

2. Quelle affectation est correcte en bash ?

   A) nom = "Awa"

   B) nom : "Awa"

   C) nom="Awa"

3. Dans un script, la variable $? contient…

   A) Le code de retour de la dernière commande

   B) Le nombre de paramètres

   C) Le PID du script

4. Pour que grep interprète +, ? et {8} comme des opérateurs de regex, on utilise…

A) grep -v

B) grep -E

C) grep -c

5. Dans awk -F';' '{print $2}', le deuxième champ d'une ligne est…

A) $0

B) $2

C) NR

6. La commande sed 's/Segou/Sikasso/' notes.txt…

A) Modifie le fichier sur place

B) Supprime Segou du fichier

C) Affiche le texte avec le premier Segou de chaque ligne remplacé, sans modifier le fichier

7. Dans une ligne crontab, « 30 3 * * 1 » signifie…

A) chaque jour à 3 h 30

B) chaque lundi à 3 h 30

C) le 1er du mois à 3 h 30

8. L'option -o pipefail de set sert à…

A) Accélérer les scripts

B) Détecter l'échec d'une commande au milieu d'un tube

C) Afficher chaque commande avant exécution

9. tar -tzf archive.tar.gz permet de…

A) Créer l'archive

B) Lister le contenu de l'archive sans l'extraire

C) Supprimer l'archive

10. Avant d'exécuter un rm construit avec une variable, la précaution indispensable est de…

A) Utiliser sudo

B) Vérifier que la variable est non vide et le chemin correct, avec les guillemets

C) Ajouter l'option -f partout

Corrigés du QCM

1. Réponse B — C'est le shebang : le système s'en sert pour choisir le programme qui exécutera le script.

2. Réponse C — Aucun espace autour du signe = : avec des espaces, bash cherche une commande nommée nom.

3. Réponse A — $? vaut 0 si la dernière commande a réussi, autre chose sinon ; le PID du script est $$, le nombre de paramètres $#.

4. Réponse B — grep -E active les expressions régulières étendues ; sans elle, ces caractères doivent être échappés en mode de base.

5. Réponse B — $2 désigne le deuxième champ (découpé ici sur les point-virgules) ; $0 est la ligne entière et NR le numéro de ligne.

6. Réponse C — Sans -i, sed écrit sur la sortie standard : le fichier est inchangé. La substitution simple remplace la première occurrence de chaque ligne (g la rendrait globale).

7. Réponse B — Champs : minute=30, heure=3, jour du mois=*, mois=*, jour de semaine=1, donc le lundi.

8. Réponse B — Sans pipefail, un tube ne rend que le code de sa dernière commande ; avec pipefail, l'échec d'un maillon fait échouer l'ensemble.

9. Réponse B — t = lister, z = gzip, f = fichier. Créer serait -czf, extraire -xzf.

10. Réponse B — Une variable vide ou mal découpée transforme rm en catastrophe ; guillemets, tests -n/-d et listage préalable sont la parade.

Résumé — les notions importantes

• Un script bash se programme comme un langage : variables sans espaces autour de =, $(( )) pour calculer, [ ] pour tester, for/while/case pour structurer.

• Les codes de retour gouvernent tout : $? dit le succès (0) ou l'échec, et if, &&, || et while ne lisent que lui.

• Administration par script : tester avant d'agir (id, -f, -d), traiter par lots en lisant les paramètres ligne par ligne, et afficher un bilan compté.

• Processus : $$ pour le PID du script, $! pour la tâche de fond, ps pour observer, kill TERM avant KILL, wait pour synchroniser.

• Texte : grep -E filtre par motif, sed remplace, awk découpe en champs et calcule ; les trois se combinent en tubes d'analyse de journaux.

• Sauvegarde fiable = archive tar horodatée + déclenchement cron + rotation find -mtime + restauration testée.

• Script de production : set -euo pipefail, guillemets systématiques, paramètres validés, droits serrés et journal horodaté de chaque action.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Le shell : variables, conditions et boucles
nom=valeur, sans aucun espace autour du = ; $nom pour lire, $(commande) pour capturer une sortie. ; Les tests [...] comparent les nombres (-ge, -lt), les chaînes (=, !=) et les fichiers (-f, -d, -s). ; for pour parcourir une liste connue, while pour répéter sous condition ou lire un fichier ligne par ligne.

Fiche 2 — Scripts d'administration : fichiers et utilisateurs
while IFS= read -r variable; do …; done < fichier : la forme sûre pour traiter un fichier ligne par ligne. ; Toujours tester avant d'agir : id pour un compte, -e ou -f pour un chemin, puis compter et afficher le bilan. ; Un script relançable sans dommage (idempotent) vaut dix scripts qu'on n'ose plus exécuter.

Fiche 3 — Gestion des processus en ligne de commande
ps -o pid,stat,cmd -p PID décrit un processus précis ; STAT S = en attente, R = en cours, Z = zombie. ; kill envoie SIGTERM par défaut (arrêt propre demandé) ; kill -9 force, sans laisser au programme le temps de nettoyer. ; wait PID attend la fin d'une tâche de fond ; le code dans $? dit si tout s'est bien passé.

Fiche 4 — Expressions régulières et traitement de texte avec grep, sed et awk
grep -E active les regex étendues (+, ?, {n} sans barre oblique) ; ^ et $ ancrent le motif en début et fin de ligne. ; sed 's/a/b/' remplace une fois par ligne, 's/a/b/g' partout ; -i écrit dans le fichier — à tester sans -i d'abord. ; awk -F';' découpe en champs $1, $2… ; le bloc END sert aux bilans (sommes, moyennes avec NR).

Fiche 5 — Planification et automatisation des sauvegardes
tar -czf / -tzf / -xzf : créer, lister, extraire. Vérifier le contenu d'une archive fait partie de la sauvegarde. ; Une ligne cron = 5 champs de temps + commande en chemin absolu ; 0 2 * * * signifie chaque jour à 2 h 00. ; Rotation avec find -mtime +N, testée d'abord sans -delete ; restauration testée avant de déclarer le système fiable.

Fiche 6 — Sécuriser ses scripts et journaliser les actions
set -euo pipefail : arrêt à la première erreur, erreur sur variable non définie, échec des tubes détecté. ; Guillemets partout, paramètres contrôlés (${1:-defaut}), scripts sensibles en chmod 700, jamais d'eval sur une saisie. ; Journaliser = écrire, avec date et heure, chaque action importante dans un fichier dédié, via une fonction unique.

Fiche 7 — Mini-projet : boîte à outils d'administrateur système
Découper en fonctions, aiguiller avec case, faire échouer proprement (usage + code 1) sur tout paramètre inconnu. ; Construire et tester action par action ; les tests d'appel (sans paramètre, paramètre faux, appels normaux) font partie du livrable. ; Documenter en tête de script et faire évoluer l'outil au fil des tâches récurrentes, sous gestion de versions.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Introduction Script Bash sur Linux

Chaîne : IT-Connect - Florian · Langue : Français · Niveau : Débutant

Une première prise en main complète du script bash : ce qu'est bash, écrire et lancer son premier script, enchaîner les commandes et utiliser les tubes. Le point de départ idéal avant les leçons 1 et 2 de ce cours.

https://www.youtube.com/watch?v=gBCGeLjYBgQ

Leçon 1 — Scripting bash : 6 - Tableaux, IFS et substitutions

Chaîne : Ordirepar · Langue : Français · Niveau : Intermédiaire

Un épisode avancé de la série Scripting bash d'Ordirepar : tableaux, rôle précis de la variable IFS dans la lecture de fichiers et substitutions de variables et de commandes. À regarder après la leçon 2 pour consolider les lectures ligne par ligne.

https://www.youtube.com/watch?v=MjaBXSKRv-U

Leçon 4 — On faisait comment avant Excel ??

Chaîne : Chaine-Archive · Langue : Français · Niveau : Intermédiaire

Manipuler des données (journaux, CSV) en ligne de commande avec awk : filtrage, transformation et agrégation en flux, avec des cas concrets où awk surpasse le tableur. Le prolongement direct et motivant de la leçon 4.

https://www.youtube.com/watch?v=nFVNBBgoTNY

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• GNU Bash Reference Manual (gnu.org/software/bash/manual) — la référence complète et gratuite du shell utilisé dans tout le cours.

• Pages de manuel locales : man bash, man grep, man sed, man awk, man tar, man crontab — disponibles hors ligne sur toute machine Linux.

• Documentation Ubuntu-fr (doc.ubuntu-fr.org) : fiches « script », « cron » et « tar » en français, avec exemples commentés.

• Tutoriels Bash d'IT-Connect (it-connect.fr) — articles et vidéos en français sur les boucles, les conditions et l'automatisation.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !